

团队汇报执行手册（中文完整版）

美国国内航班同机延误传播项目（2015-2024）

这份文档是给“没参与具体开发/计算的队友”准备的。

目标是让任何人都能看懂：

1. 我们到底做了什么；
 2. 为什么要这么分任务和跑 CHTC；
 3. 每一步产出了什么；
 4. 结果怎么讲到 slides 里。
-

0. 一句话项目结论（先看这个）

我们用 2015-2024 年 BTS 月度航班数据，按同一架飞机（TAIL_NUM）连接前后航段后发现：

- 前序航班晚到与下一段“严重晚点（出发晚点 > 15 分钟）”风险显著正相关；
 - 平均每 +10 分钟前序晚到，对应下一段严重晚点赔率约 **1.45 倍**；
 - 过站时间越短，传播越强（20-44 分钟的 carry-over 最强）。
-

1. 项目目标与分析范围

1.1 研究问题

当前一段航班晚到后，延误会传递到同一架飞机后续航班吗？传递强度多大？在哪些场景更明显？

1.2 最终主线（汇报采用）

- 主结果：logistic（风险视角）
- 辅助：linear（分钟数视角）
- 异质性：只讲两个维度
 - 航司差异
 - 过站时间差异（turnaround bin）

1.3 时间范围

- 2015-01 到 2024-12（10 年，120 个月文件）
-

2. 数据与规模（队友必须知道的数字）

数据源：

- U.S. Bureau of Transportation Statistics (BTS) On-Time Performance 月度数据

规模：

- 原始压缩包：约 **3.19GB** (stat605_2015_2024_raw.tar)
- 清洗后分区分配总行数：约 **61,690,070**
- 最终建模 flight pairs: **53,268,469**
- 仅 2024 年 pairs: **6,021,015**

说明：

- 原始 CSV 不修改；
- 不合并成一个超大 raw 文件；
- 全流程按阶段写中间产物，保证可复现。

3. 变量与构造（讲方法时可直接引用）

3.1 原始核心字段

- TAIL_NUM
- FL_DATE
- OP_UNIQUE_CARRIER
- ORIGIN, DEST
- CRS_DEP_TIME, CRS_ARR_TIME
- DEP_DELAY, ARR_DELAY
- DISTANCE
- CANCELLED, DIVERTED

3.2 衍生字段（清洗/配对后）

- sched_dep_hour, sched_arr_hour
- sched_dep_timestamp, sched_arr_timestamp（含跨夜修正）
- route = ORIGIN-DEST
- scheduled_turnaround_minutes
- same_airport_connection = 1(prev_DEST == current_ORIGIN)
- current_dep_delayed_15 = 1(current_DEP_DELAY > 15)
- prev_arr_delayed_15 = 1(prev_ARR_DELAY > 15)
- turnaround_bin:
 - 20-44
 - 45-89
 - 90-179
 - 180-600

4. 我们实际做了什么（按执行顺序）

这一节是“全流程流水账 + 目的解释”，给队友看最有用。

Step A: 准备与清洗（按月并行）

做法：

- 每个月一个 cleaning job；
- 去掉关键字段缺失、取消、备降；
- 生成标准时间字段（含跨夜修正）。

为什么这样做：

- 月度并行最自然，单任务体量可控；
- 后续 CHTC 可以高并发提交，失败也容易重跑。

Step B: 全样本按 tail 分区（不是按年分区）

做法：

- 读取全部 cleaned months；
- 用 `hash(TAIL_NUM) % 100` 分成 100 个 partition；
- 保证同一架飞机永远在同一个 partition。

为什么这样做：

- 防止同一飞机跨年相邻航段被拆散；

- 支持后续按 partition 并行构 pair。

Step C: 构建连续航段 pairs (按 partition 并行)

做法:

- 每个 partition 一个 job;
- 按 TAIL_NUM + sched_dep_timestamp 排序;
- 当前航班配对前一航班;
- 只保留:
 - prev_DEST == current_ORIGIN
 - 20 <= scheduled_turnaround_minutes <= 600

为什么这样做:

- 这是同机延误传播最核心定义;
- 去掉不合理连接, 避免噪声配对。

Step D: 先跑主模型 (稳定优先)

做法:

- 一开始试过大窗口 pooled job (3 年一组/全量), 多次遇到 OOM held;
- 改为“按年跑主模型”(10 个 job), 显著更稳;
- 对重负载年份单独提内存补跑 (2018/2019/2024)。

为什么这样做:

- CHTC 上高内存可用槽位有限;
- 年度任务更容易排队和恢复;
- 先保证主结果完整, 比一次性全量失败更可靠。

Step E: 单独补跑 2024 异质性

做法:

- 单独提交 2024 heterogeneity job;
- 跑 airline 与 turnaround 两个交互模型;
- 产出异质性图表。

为什么这样做:

- 异质性模型更吃内存与时间;
- 拆开跑可降低主流程失败风险;
- 汇报里用 2024 做异质性展示足够清晰。

Step F: 结果回传本地并制图打包

做法:

- 用 rsync 把 CHTC 结果拉回本地;
- 统一生成 results/report_figures 图包;
- 生成英文/中文文档 (HTML+PDF)。

为什么这样做:

- 避免现场演示时远端依赖;
- 所有人直接拿本地成品文件做 slides。

5. CHTC 是怎么跑的 (新手版)

5.1 登录与目录

```
ssh yli2827@ap2001.chtc.wisc.edu
cd ~/stat605_delay_pipeline
```

5.2 查看任务状态

```
condor_q
condor_q -hold
condor_q <cluster_id> -nobatch
```

5.3 查看 held 原因

```
condor_q -hold
condor_q <cluster_id>.<proc_id> -better-analyze
```

我们最常见 hold:

- 34/102: cgroup memory 超限 (OOM)

5.4 主模型任务提交 (按年)

使用:

- chtc/fit_delay_models_years_2015_2024_main.sub

特点:

- 一年一个 job (共 10 个)
- 稳定性高于大窗口 pooled job

5.5 重负载年份补跑

使用:

- chtc/fit_delay_models_retry_2018_2019_2024.sub

特点:

- 只重跑失败年份;
- 提高内存, 避免整批重来。

5.6 2024 异质性提交

使用:

- chtc/fit_delay_models_2024_hetero.sub

特点:

- 单年单任务;
- 专门跑交互项 (airline / turnaround)。

6. 最终结果 (可直接搬到 slides)

6.1 年度主结果表 (2015-2024)

年份	Pairs	Logistic OR (每 +10 分钟)	Linear 系数	每 +10 分钟传递的分钟数
2015	4,956,280	1.525	0.5212	5.21
2016	4,824,439	1.500	0.5226	5.23
2017	4,862,737	1.491	0.5297	5.30
2018	6,201,714	1.469	0.5274	5.27
2019	6,380,395	1.439	0.5190	5.19
2020	3,575,671	1.453	0.4221	4.22
2021	4,943,061	1.414	0.4922	4.92
2022	5,620,042	1.416	0.5285	5.28
2023	5,883,115	1.405	0.5180	5.18
2024	6,021,015	1.403	0.5266	5.27

总览:

- 总 pairs: **53,268,469**
- 平均 OR (每 +10 分钟): **1.451**
- 平均分钟传递 (每 +10 分钟): **5.11 分钟**

6.2 2024 过站异质性 (最强信号)

Turnaround bin	OR (每 +10 分钟)
20-44	2.859
45-89	2.132
90-179	1.315
180-600	1.059

解释:

- 过站越短, 延误传播越强。

7. 图怎么用 (汇报推荐顺序)

图目录:

- /Users/liyuang/Desktop/stat605/results/report_figures

推荐主线:

1. fig04_pipeline_parallel_flow.png (先讲流程)
2. fig08_pairs_by_year_2015_2024.png (讲规模)
3. fig06_yearly_or10_trend_2015_2024.png (讲主结果)
4. fig01_delay_rate_by_prev_delay_bin_2024.png (讲非参数趋势)
5. fig07_yearly_linear_minutes_trend_2015_2024.png (讲分钟解释)
6. fig03_carryover_by_turnaround_bin_2024.png (讲异质性主图)
7. fig02_carryover_by_airline_2024.png (补充异质性)

8. 7 页 PPT 模板 (队友直接照抄)

Slide 1: 研究问题 + 意义

- 同机前序晚到是否传播到下一段?
- 为什么对运营和旅客很重要?

Slide 2: 数据与规模

- 2015-2024, 120 月文件
- 53.27M pairs
- 放 fig08

Slide 3: 方法与并行流程

- 清洗、分区、配对逻辑
- 放 fig04

Slide 4: 主 logistic 结果

- OR per 10 min
- 放 fig06 + fig01

Slide 5: linear 分钟解释

- “10 分钟前序晚到 -> 平均多晚出发多少分钟”
- 放 fig07

Slide 6: 异质性

- turnaround (主)
- airline (辅)
- 放 fig03 + fig02

Slide 7: 局限与结论

- association 不是 strict causality
 - 天气/ATC/机械未完全控制
 - 运营建议: 重点保护短过站 buffer
-

9. 讲解时必须注意 (避免翻车)

- 不要说“causal effect”, 要说“associated with / 相关”。
 - 不要把航司排序讲成“绝对优劣”, 要加样本量背景。
 - 不要把 2024 异质性讲成“10 年都验证过”。
 - 不要把线性模型 R^2 当主卖点, 主卖点是 logistic 风险传播。
-

10. 你可以直接念的收尾 (30-40 秒)

“我们基于 2015 到 2024 年 BTS 数据, 构建了超过 5300 万个同机连续航段配对。结果显示, 前序航班晚到与下一段严重晚点风险稳定正相关: 平均每增加 10 分钟前序晚到, 下一段严重晚点赔率约提高到 1.45 倍。在线性解释上, 这相当于平均多传递约 5 分钟出发延误。异质性结果表明短过站场景传播最强, 因此在运行管理上, 保护短过站 buffer 是最直接可操作的优化方向。”

11. 文档与成品文件路径 (直接发给队友)

- 本文 Markdown:
 - /Users/liyuang/Desktop/stat605/results/report_figures/team_slides_playbook_zh.md
- 本文 HTML:
 - /Users/liyuang/Desktop/stat605/results/report_figures/team_slides_playbook_zh.html
- 本文 PDF:

– /Users/liyuan/Desktop/stat605/results/report_figures/team_slides_playbook_zh.pdf